

Automated Detection of Thoracic Pathologies from Chest X-ray Images

Deep Learning Project Report

Submitted by: 23f2000575

1. Introduction and Problem Statement

In this project, I worked on building an automated deep learning system that can detect thoracic diseases from chest X-ray images. This is a real and important problem in the medical domain because chest X-rays are one of the most common diagnostic tools used by doctors to detect lung infections, fluid accumulation, and structural abnormalities of the heart and lungs. However, having a radiologist manually analyze each X-ray is time-consuming and can be inconsistent due to human error and fatigue. An automated system that can assist doctors in this process would be very valuable.

The task is formulated as a **multi-class classification problem**. This means that given a chest X-ray image, my model must assign it to exactly one of **20 possible pathology classes**. The classes include a wide range of thoracic conditions such as Atelectasis, Cardiomegaly, Consolidation, Edema, Effusion, Emphysema, Fibrosis, Hernia, Infiltration, Mass, Nodule, Pleural Thickening, Pneumonia, Pneumothorax, Pneumoperitoneum, Pneumomediastinum, Subcutaneous Emphysema, Tortuous Aorta, Calcification of the Aorta, and No Finding. The No Finding class represents cases where no abnormality is detected in the X-ray.

1.1 Dataset

I was provided with a dataset of chest X-ray images organized as follows:

- **Training set:** 51,043 images with labels provided in `train.csv`. Each row in the CSV contains the image ID and 20 binary columns corresponding to the 20 pathology classes. Since each image belongs to exactly one class, exactly one column has a value of 1 and all others are 0.
- **Test set:** 17,015 images without labels, provided for generating Kaggle submission predictions.
- **Images:** All images are 384×384 pixels in RGB format. However, as I discovered during EDA, the images are actually grayscale X-rays where the same pixel values are repeated across all three RGB channels.

1.2 Evaluation Metric

The competition used a **macro-averaged asymmetric cost function**, which is different from standard accuracy metrics. The score for each class is computed as:

$$\text{Score}_c = \frac{TP_c - FP_c - 5 \cdot FN_c}{N_c}$$

Where TP is True Positives, FP is False Positives, FN is False Negatives, and N is the total number of true samples for that class. The final score is the average of this across all 20 classes.

What makes this metric particularly challenging is the **asymmetric penalty**. A False Negative, meaning the model misses a disease, is penalized 5 times more heavily than a False Positive. In a medical context, this makes complete sense. Missing a disease in a patient is far more dangerous than raising a false alarm. This

asymmetric scoring mechanism strongly pushed me to build a model that is sensitive to detecting diseases, not just one that achieves high accuracy by predicting the majority class.

Additionally, the use of macro-averaging means every class, including rare ones, contributes equally to the final score. This prevents the model from simply ignoring rare diseases and predicting the dominant class all the time.

2. Exploratory Data Analysis (EDA)

Before jumping into model building, I spent time understanding the data through exploratory data analysis. This is a critical step that helped me understand what I am working with and what challenges I need to address.

2.1 Image Exploration

I started by loading and visualizing the first image in the training set. I used the Python Imaging Library (PIL) to open the image and then converted it to a NumPy array for inspection. The image had a size of 384×384 pixels and was in RGB mode. I printed the shape of the NumPy array, which confirmed it was `(384, 384, 3)`, meaning height 384, width 384, and 3 color channels.

One interesting thing I explored was whether the three color channels were actually carrying different information. Since these are X-ray images, which are inherently grayscale, I suspected that the RGB format was just a result of how the images were saved and that all three channels would be identical. I verified this by extracting the Red, Green, and Blue channels separately and comparing them using `numpy.array_equal`. The result confirmed that all three channels were indeed identical, meaning the images are effectively grayscale images stored in RGB format.

2.2 Class Distribution Analysis

Next, I analyzed the distribution of the 20 disease classes in the training set. I summed up the binary columns in the training CSV to count how many images belong to each class. I then sorted these counts in descending order and also computed the percentage of each class.

The results revealed a **severe class imbalance**, which I had already expected given the nature of medical datasets. The dominant class was **No Finding** with 34,079 images, representing about **66.77%** of the entire training set. This makes sense because in a real population, most X-rays taken do not show a serious disease. On the other end of the spectrum, rare diseases like Pneumomediastinum had only **5 samples** (0.01%), Subcutaneous Emphysema had 24 samples (0.05%), and Hernia had only 37 samples (0.07%).

I visualized this class distribution using a bar chart, which made the imbalance visually very obvious. The bar for No Finding was towering above all other classes.

This imbalance has serious implications for model training. A naive model that always predicts No Finding would achieve about 67% accuracy, which sounds decent but is practically useless. More importantly, given the competition's scoring metric that heavily penalizes False Negatives, such a model would score very poorly because it would miss almost all actual disease cases and accumulate massive penalties of -5 for each missed disease. I made a mental note that handling this class imbalance carefully was going to be one of the most important aspects of this project.

3. Data Preprocessing and Augmentation

3.1 The Need for Preprocessing

Before feeding images into a deep learning model, I needed to preprocess them into a standardized format that the model can work with. Neural networks expect inputs as numerical tensors of a fixed size, and they also perform better when the input values are normalized to a standard range. Additionally, the model I planned to use (more on this later) was pre-trained on the ImageNet dataset, so I needed to apply the same normalization that was used during its original training.

3.2 Training Transforms

For the training set, I applied the following sequence of transformations:

- 1. Resize to (224, 224):** The original images are 384×384 pixels. I resized them to 224×224 because the pre-trained model I used was originally designed to work with this input size. Resizing also ensures a fixed and uniform input size for all images.
- 2. RandomHorizontalFlip:** This transformation randomly flips the image horizontally with a 50% probability during training. The purpose of this is data augmentation. A flipped chest X-ray still looks like a valid chest X-ray (left-right flips in chest images are generally acceptable from a medical perspective). By randomly flipping, I am effectively doubling the diversity of the training data without collecting new images. This helps the model generalize better and reduces overfitting.
- 3. RandomRotation(10):** This applies a random rotation of up to ± 10 degrees to the image. In practice, X-rays taken in a clinical setting may have slight variations in how the patient is positioned. By introducing small random rotations during training, I am teaching the model to be robust to these small positional variations. A rotation of only 10 degrees is a deliberate choice since too large a rotation could distort the anatomy and confuse the model.
- 4. ToTensor:** This transformation converts the PIL Image (which has pixel values from 0 to 255) into a PyTorch tensor with values ranging from 0.0 to 1.0. It also reorders the dimensions from (Height, Width, Channels) to (Channels, Height, Width), which is the format that PyTorch expects for image data.
- 5. Normalize with ImageNet mean and standard deviation:** I applied normalization using the mean `[0.485, 0.456, 0.406]` and standard deviation `[0.229, 0.224, 0.225]` for the three channels. These specific values come from the statistics of the ImageNet dataset, which is the dataset on which my pre-trained backbone was originally trained. By normalizing the input with these same statistics, I ensure that the input distribution my model sees during fine-tuning is similar to what it saw during pre-training. This is a standard and important step when using transfer learning, as it helps the pre-trained weights remain effective without needing to be drastically re-adjusted in the early layers.

3.3 Validation and Test Transforms

For the validation and test sets, I applied a simpler pipeline without any random augmentations. I only applied Resize, ToTensor, and Normalize. The reason I did not apply random flips or rotations here is that during evaluation and inference, I want deterministic and consistent predictions. Augmentation is only for training to help the model generalize. Applying random augmentations during testing would give different results each time I run the evaluation, which is undesirable.

3.4 The Data Augmentation Mindset

The overall philosophy behind my augmentation strategy was to simulate the natural variability that exists in real-world chest X-ray data without distorting the clinically meaningful features. I deliberately kept the augmentations mild. I did not apply color jitter since these are grayscale images and color changes would be meaningless. I did not apply extreme rotations or cropping since that could remove important anatomical structures from the image. The goal was to add enough variety to improve generalization while keeping the images medically plausible.

4. Dataset and DataLoader

4.1 Custom Dataset Class

I implemented a custom PyTorch Dataset class called `ChestXrayDataset`. This class takes a DataFrame (containing image IDs and labels), the image directory path, and the transform pipeline as inputs. In the `__getitem__` method, it loads each image from disk by constructing the file path from the image ID, opens it using PIL, converts it to RGB, and applies the transform. To extract the label, it reads the 20 binary columns and uses `argmax` to find which column has the value 1, converting the one-hot encoding into a single integer class index. This integer label is what the loss function expects during training.

4.2 Train-Validation Split

I split the training data into an 80-20 train-validation split using `train_test_split` from scikit-learn. A critical detail here was using **stratified splitting** with the `stratify` parameter. Since the dataset is severely imbalanced, a random split could accidentally put all samples of a rare class into one split and leave the other split with no samples of that class at all. Stratified splitting ensures that the class distribution is preserved proportionally in both the training and validation sets. This gave me 40,834 training samples and 10,209 validation samples.

4.3 DataLoaders

I created PyTorch DataLoader objects for both the training and validation datasets with a batch size of 32. For the training loader, I set `shuffle=True` so that the model sees the data in a different order each epoch, which helps training stability. For the validation loader, I set `shuffle=False` since order does not matter for evaluation. I also used `pin_memory=True` when on Kaggle (where a GPU is available) to speed up the data transfer from CPU to GPU memory.

4.4 Test Dataset and DataLoader

I also implemented a separate `ChestXrayTestDataset` class for the test data. The key difference from the training dataset is that there are no labels in the test set. Instead of returning a label, this dataset returns the image tensor along with the image ID string. This is necessary so that after generating predictions, I can match each prediction back to its image ID and create the correct submission CSV.

5. Model Building

5.1 Starting with a Simple CNN from Scratch

To understand the baseline and to satisfy the academic requirement of exploring multiple approaches, I started by designing a simple Convolutional Neural Network (CNN) from scratch. This network consisted of three convolutional blocks, each containing a 2D convolutional layer followed by a ReLU activation function and a MaxPooling layer to reduce spatial dimensions. Specifically, the first convolutional layer had 32 filters of size 3×3, the second had 64 filters, and the third had 128 filters. After the convolutional blocks, I flattened the feature maps and passed them through two fully connected layers, the first with 256 neurons and a ReLU activation, and the second being the output layer with 20 neurons corresponding to the 20 classes. I used dropout with a rate of 0.5 after the first fully connected layer to reduce overfitting.

However, as expected, the performance of this simple CNN was very poor. A model with this level of capacity, trained from random weight initialization, does not have the ability to learn the subtle and complex features present in medical X-ray images. The training was unstable and the competition score on the validation set was very bad. This result was expected and it motivated me to move on to a much more powerful approach.

5.2 Transfer Learning with DenseNet-121

After the poor results with the simple CNN, I decided to use **transfer learning** with a pre-trained DenseNet-121 model. Transfer learning is the principle of taking a model that has already been trained on a very large dataset for a related task and fine-tuning it for a new, more specific task. The intuition is that the model has already learned useful low-level features like edges, textures, and shapes during its original training, and these features are useful for the new task as well.

Why DenseNet-121?

I chose DenseNet-121 specifically for several strong reasons:

First, **DenseNet** (Densely Connected Convolutional Networks) has a unique architectural property where each layer is connected to every subsequent layer in a feed-forward fashion [1]. This means the feature maps from all preceding layers are concatenated together and passed to the next layer. This dense connectivity encourages feature reuse across the network and allows gradients to flow more easily during backpropagation, which mitigates the vanishing gradient problem and results in very efficient training.

Second, DenseNet-121 is **lightweight** compared to other powerful architectures like ResNet-50 or VGG-16. It achieves competitive performance with significantly fewer parameters, which is important when training resources are limited.

Third, and most importantly, DenseNet-121 has been shown to be particularly effective for **chest X-ray analysis**. The landmark CheXNet paper [2] demonstrated that a DenseNet-121 model could detect pneumonia from chest X-rays at a level that exceeded radiologist performance. This gave me strong domain-specific confidence that DenseNet-121 was the right architectural choice for this task.

I loaded the DenseNet-121 model with its default pre-trained weights from ImageNet using PyTorch's `torchvision.models` library.

5.3 Architecture of the Modified DenseNet-121

The original DenseNet-121 was designed for ImageNet's 1,000-class classification task. I needed to adapt it for my 20-class medical classification task. I did this by replacing the final classification head.

The original classifier of DenseNet-121 is a single linear layer that maps 1,024 input features to 1,000 output classes. I replaced this with a custom classification head consisting of:

1. **Linear layer (1024 → 512):** This fully connected layer takes the 1,024-dimensional feature vector produced by the DenseNet backbone and projects it down to 512 dimensions. This allows the model to learn a compressed, task-specific representation from the rich features extracted by the backbone.
2. **ReLU activation:** A Rectified Linear Unit activation function is applied after the first linear layer to introduce non-linearity. Without non-linear activations, stacking multiple linear layers is mathematically equivalent to a single linear layer, so ReLU is essential for the model to learn complex decision boundaries.
3. **Dropout (p=0.3):** I applied dropout with a probability of 0.3. This randomly sets 30% of the neurons to zero during training, which acts as a regularization technique to prevent the classification head from overfitting to the training data. A rate of 0.3 was chosen as a moderate value, enough to regularize without overly restricting the model's capacity.
4. **Linear layer (512 → 20):** The final output layer maps the 512-dimensional representation to 20 output logits, one for each of the 20 pathology classes. During training, these raw logits are passed to the CrossEntropy loss function, and during inference, I take the argmax to get the predicted class.

The total parameter count of this modified model came out to approximately **7.49 million parameters**, all of which were trainable. This is a relatively compact model, which was an advantage given the computational constraints of the Kaggle environment.

5.4 Multi-GPU Support

I also added support for multi-GPU training using PyTorch's `DataParallel`. The code checks if more than one GPU is available and, if so, wraps the model in `nn.DataParallel`. On Kaggle, I had access to 2 GPUs, and the notebook output confirmed that `DataParallel` was applied, which helped speed up training by splitting each batch across the two GPUs.

6. Loss Function and Optimizer

6.1 Loss Function

I used **CrossEntropyLoss** as my primary loss function. CrossEntropyLoss combines a Softmax activation and Negative Log Likelihood Loss into a single operation and is the standard loss function for multi-class classification tasks.

I am aware that given the severe class imbalance and the asymmetric penalty in the competition metric, more sophisticated loss functions could potentially improve performance. In particular, **Focal Loss** [3] is a modification of Cross Entropy that down-weights the loss contribution from easy, well-classified examples (mostly the dominant No Finding class) and focuses the model's learning on hard, misclassified examples (the rare disease classes). This would have been a natural next step to explore had I continued iterating on the model.

Similarly, class-weighted Cross Entropy, where the loss for each class is multiplied by a weight inversely proportional to its frequency, would have been a direct way to address the class imbalance during training.

6.2 Optimizer

I used the **Adam optimizer** with a learning rate of `1e-4`. Adam is an adaptive learning rate optimizer that combines the ideas of momentum and RMSProp. It adapts the learning rate individually for each parameter based on the first and second moments of the gradients. It is widely regarded as a reliable default choice for fine-tuning pre-trained models because its adaptive nature allows it to find a good solution without requiring extensive learning rate tuning.

7. Model Training

7.1 Training Loop

I organized the training logic into clean, modular functions:

- **train_one_epoch**: This function runs one full pass over the training data. For each batch, it moves the images and labels to the GPU, performs a forward pass through the model to get predictions, computes the CrossEntropy loss, runs backpropagation to compute gradients, and then updates the model weights using the Adam optimizer. It accumulates all predictions and labels and returns the average loss along with the predictions and true labels for the entire epoch.
- **evaluate**: This function runs the model in evaluation mode over the validation data loader. It uses `torch.no_grad()` to disable gradient computation, which saves memory and speeds up inference. It collects all predictions and returns them for scoring.
- **run_training**: This is the main training orchestrator. It runs the training for a total of **10 epochs**, calling `train_one_epoch` and then `evaluate` at the end of each epoch. After each epoch, it computes the competition score on both the training and validation sets and prints the results.

7.2 Best Model Checkpointing

One important practice I followed was saving the best model checkpoint. After each epoch, I compared the current validation competition score with the best score seen so far. If the current score was better, I saved the model weights to a checkpoint file at `models/best_model.pth`. This ensures that even if the model starts to overfit in later epochs, I always have the best-performing version of the model saved and can load it for generating final predictions.

7.3 Early Stopping

I also implemented early stopping with a patience of 5. This means that if the validation score did not improve for 5 consecutive epochs, I stopped the training early. This prevents wasting computation on training that is no longer improving the model and also acts as another form of regularization against overfitting.

8. Generating Predictions and Submission

8.1 Prediction Generation

After training was complete, I loaded the best saved model checkpoint from `models/best_model.pth` using `torch.load` with `map_location=device`. I then ran the model on the full test dataset using the

`generate_predictions` function.

This function runs the model in evaluation mode over all 532 test batches. For each batch, it gets the model's output logits, takes the argmax to determine the predicted class index, and stores the image ID and predicted class index together. Progress was printed every 50 batches so I could monitor the prediction generation.

After collecting all predictions, the function constructs a one-hot encoded DataFrame where each row has the image ID and exactly one column set to 1 (the predicted class) and all others set to 0.

8.2 Submission CSV

I saved the predictions to `submission.csv`. The final submission DataFrame had a shape of **(17,015 rows × 21 columns)**, which matches the expected format exactly: one row per test image, one ID column, and 20 binary class columns. I also printed the head of the DataFrame to visually verify that the format was correct before submitting to Kaggle.

I then uploaded this submission CSV to the Kaggle competition and received a score on the public leaderboard, which evaluates on 65% of the test set.

9. Conclusion and Learnings

This project was a very valuable learning experience that exposed me to the full end-to-end pipeline of building a deep learning system for a real-world medical imaging task.

Key learnings from this project:

1. **Class imbalance is a fundamental challenge in medical AI.** The No Finding class dominated the dataset at nearly 67%. Simply ignoring this and training a naive model leads to a classifier that is biased toward the majority class. I learned that this must be addressed carefully, either through class-weighted loss, oversampling, or specialized loss functions like Focal Loss.
2. **The choice of evaluation metric deeply influences model design.** The asymmetric scoring metric with heavy penalties for False Negatives changed how I thought about the problem. Optimizing for standard accuracy would have been completely misaligned with the competition objective. This taught me to always analyze the evaluation metric carefully before designing the model.
3. **Transfer learning is extremely powerful, especially in low-data regimes for rare classes.** Training a model from scratch on this dataset, especially with the severe class imbalance, would have been very challenging. Using a pre-trained DenseNet-121 allowed me to start from a much stronger initialization and leverage features already learned from millions of images.
4. **Engineering discipline matters.** Practices like saving the best model checkpoint, implementing early stopping, using stratified splitting, and separating training and validation transforms are not just good habits; they directly impact the quality of the final result.
5. **Data augmentation must be domain-aware.** I learned that augmentation choices in medical imaging must be made carefully. Not all augmentations that work for natural images are appropriate for medical images. For example, vertical flips in chest X-ray images would produce anatomically unrealistic images (lungs appearing below the stomach) and could confuse the model.

References

- [1] Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). **Densely Connected Convolutional Networks**. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4700–4708.
- [2] Rajpurkar, P., Irvin, J., Ball, R. L., Zhu, K., Yang, B., Mehta, H., ... & Ng, A. Y. (2017). **CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning**. *arXiv preprint arXiv:1711.05225*.
- [3] Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). **Focal Loss for Dense Object Detection**. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2980–2988.
- [4] Wang, X., Peng, Y., Lu, L., Lu, Z., Bagheri, M., & Summers, R. M. (2017). **ChestX-ray8: Hospital-scale Chest X-ray Database and Benchmarks**. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2097–2106.